

Bloque 04 - NumPy y cálculo vectorizado

Propósito

En este bloque Leo trabaja con el caso continuo de **NexoCloud**, una aplicación que registra para cada día las solicitudes resueltas, los minutos de respuesta y el canal de entrada. NumPy es una biblioteca de Python para guardar números en un **array** y calcular con muchos valores de una vez. No sustituye a saber qué mide cada número: hace más segura y breve la parte repetitiva cuando el significado ya está definido.

Al terminar podrás transformar una matriz de métricas operativas, seleccionar observaciones, detectar valores faltantes y explicar qué dimensión representa cada resultado. Es la base numérica sobre la que el bloque 05 construirá tablas con nombres de columnas usando Pandas.

Resultados observables

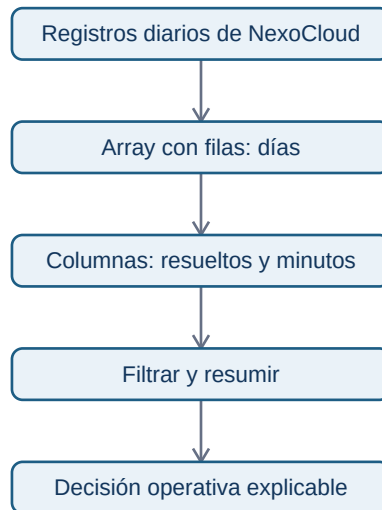
- Crear arrays, leer su dtype, ndim, shape y tamaño sin confundirlos con su significado de negocio.
- Aplicar operaciones vectorizadas y reducciones por eje, comprobando unidades y denominadores.
- Extraer subconjuntos con índices, cortes y máscaras booleanas sin desalinear datos.
- Usar broadcasting conscientemente y reconocer el error de forma antes de que propague una regla equivocada.
- Tratar NaN, distinguir una vista de una copia y reproducir una simulación sencilla.

Prerrequisitos y forma de trabajo

Se asume el vocabulario básico de Python del bloque 02: variable, lista, función y error. No se asume experiencia previa con bibliotecas. Puedes leer todo desde el móvil; para ejecutar el laboratorio usa un navegador con Google Colab o un entorno Python remoto. El script está en notebooks/practicas/04-operaciones-nexocloud.py.

Mapa del caso

La pregunta es: «¿qué canal requiere atención esta semana sin confundir días, métricas o datos ausentes?».



El array contiene posición; el diccionario de métricas del caso aporta significado. Mantendremos ambos durante todo el bloque.

Lecciones

1. Arrays, tipos y cálculo vectorizado
2. Forma, ejes e indexación
3. Máscaras, valores ausentes y copias
4. Broadcasting y reglas por canal
5. Simulación, reproducibilidad y laboratorio

Práctica evaluable

Resuelve el [diagnóstico operativo de NexoCloud](#) antes de consultar la [solución razonada](#). La práctica no pide memorizar sintaxis: pide justificar formas, filtros, datos faltantes y una recomendación.

Arrays, tipos y cálculo vectorizado

Objetivos y prerequisites

Al terminar sabrás convertir valores numéricos en un array, comprobar el tipo que NumPy ha elegido y aplicar una misma regla a todos los valores. Necesitas conocer variables y listas de Python. Un **dato** es un valor que describe algo; una **colección** agrupa varios valores. NumPy resulta útil cuando esa

colección tiene una estructura numérica conocida.

El problema antes del nombre técnico

NexoCloud cerró 5, 8 y 6 solicitudes en sus tres primeros días de una prueba. Para calcular la carga total no hace falta recorrer cada número manualmente:

```
solicitudes = [5, 8, 6]
```

Una lista es flexible: puede mezclar texto, números y otros objetos. Para cálculo científico conviene una estructura homogénea. Un **array de NumPy** (`numpy.ndarray`) es un bloque ordenado de elementos, normalmente de un tipo compatible, sobre el que las operaciones aritméticas se aplican elemento a elemento.

```
import numpy as np

resueltas = np.array([5, 8, 6], dtype=np.int64)
minutos = np.array([42.5, 51.0, 47.5], dtype=float)
print(resueltas.dtype) # int64: enteros
print(minutos.dtype)   # float64: números con decimales
```

`dtype` significa **data type**, el tipo con que el array guarda sus elementos. No es una etiqueta de negocio: `float64` no dice si 42.5 son euros, minutos o usuarios. Esa unidad debe documentarse fuera del array.

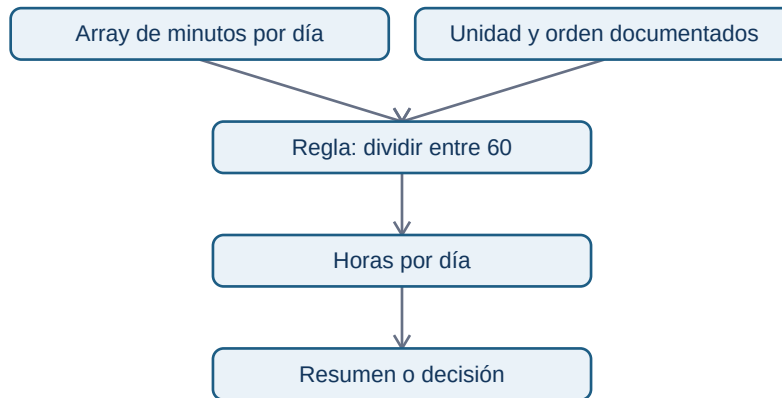
Vectorizar: una regla, muchos elementos

Supón que el equipo quiere convertir minutos a horas para comparar el esfuerzo diario. La **vectorización** aplica una operación a cada posición correspondiente sin escribir un bucle explícito:

```
horas = minutos / 60
objetivo_minutos = np.array([45.0, 45.0, 45.0])
desviacion = minutos - objetivo_minutos
```

El resultado de `minutos / 60` conserva tres posiciones. La tercera línea compara día con día porque los arrays tienen la misma longitud y el mismo orden. Es más fácil revisar «esta regla se aplica a todos los días» que repetir una instrucción manual por cada valor.

Este diagrama responde a «¿qué tiene que seguir siendo cierto para que una operación vectorizada signifique lo que creemos?»:



La flecha desde unidad y orden no es decorativa: sin ellos el cálculo puede ejecutarse y seguir siendo analíticamente falso.

Reducciones: de muchos valores a un resumen

Una **reducción** resume varias posiciones en un resultado. `sum`, `mean`, `min` y `max` son comunes:

```
total = resueltas.sum()      # 19 solicitudes
media = minutos.mean()      # 47.0 minutos
peor_dia = minutos.max()    # 51.0 minutos
```

La media no demuestra que cada cliente espere 47 minutos. Es una descripción de esos tres días y puede ocultar picos. Antes de comunicar «el servicio tarda 47 minutos», define el período, la población incluida y si una media es la medida adecuada.

Conversión y pérdida de información

NumPy intenta encontrar un tipo común. Si introduces un decimal entre enteros, puede convertir todo el array a `float`; si introduces texto, puede convertir los números en texto. Forzar un tipo también puede perder información:

```
np.array([42.9, 51.0], dtype=int) # array([42, 51]); trunca, no redondea
```

No uses esta conversión para «limpiar» datos sin investigarlos. Truncar minutos puede sesgar un promedio y convertir identificadores como "0012" a entero puede borrar ceros significativos. Un identificador no es una magnitud para sumar.

Resumen y comprobación

- Un array organiza valores con forma y tipo; la unidad de negocio no viaja sola en `dtype`.

- Vectorizar aplica una regla a cada elemento; presupone orden, unidad y población compatibles.
- Las reducciones resumen, pero no explican por sí solas la distribución.

Comprueba: ¿por qué `np.array([1, "2"])` es peligroso para una suma? ¿Qué hipótesis estás haciendo al restar dos arrays de igual longitud? En la siguiente lección verás cómo expresar qué representa cada dimensión.

Forma, ejes e indexación

Objetivos y prerequisites

Aprenderás a leer una matriz como filas y columnas, resumir por la dirección correcta y seleccionar posiciones sin alterar su significado. Requiere arrays y operaciones vectorizadas del tema anterior.

Una matriz necesita un contrato

Una lista de números no indica por sí sola qué representa cada posición. En NexoCloud registraremos cuatro días (filas) y tres canales (columnas: web, chat y correo). Cada celda es el número de solicitudes resueltas ese día por ese canal:

```
import numpy as np

canales = np.array(["web", "chat", "correo"])
resueltas = np.array([
    [12, 8, 4],
    [15, 7, 5],
    [11, 9, 6],
    [13, 10, 4],
])
print(resueltas.shape) # (4, 3): 4 días, 3 canales
print(resueltas.ndim) # 2 dimensiones
```

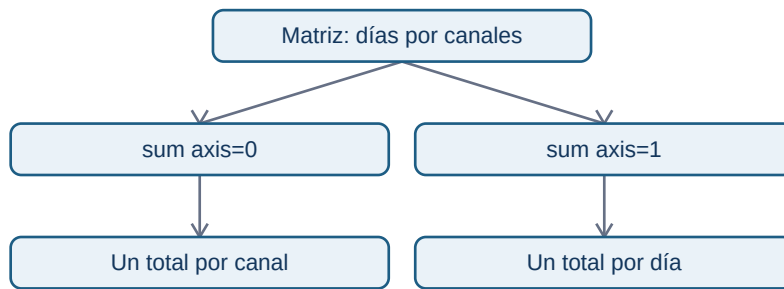
`shape` es una tupla que enumera el tamaño de cada dimensión. No dice «días» ni «canales»: eso es parte del contrato que acabamos de escribir. Una matriz con forma `(4, 3)` también podría significar cuatro tiendas y tres productos. `ndim` cuenta dimensiones y `size` cuenta celdas: aquí $4 * 3 = 12$.

Ejes: resumir hacia una dirección

En una matriz bidimensional, `axis=0` reduce las filas y conserva columnas; `axis=1` reduce las columnas y conserva filas. Es más seguro describirlo como «lo que queda» que memorizar una frase:

```
por_canal = resueltas.sum(axis=0) # [51, 34, 19], un total por canal
por_dia = resueltas.sum(axis=1) # [24, 27, 26, 27], un total por día
```

La relación visual responde a «¿qué etiqueta conserva cada suma?»:



`axis=0` no significa «mejor» ni «vertical» en abstracto; solo resulta correcto porque el contrato asignó filas a días y columnas a canales. Verifica la forma del resultado antes de asignarle un nombre.

Índices y cortes

Python empieza a contar en cero. La sintaxis `[fila, columna]` selecciona una celda; los cortes `inicio:fin` incluyen el inicio y excluyen el final:

```
primer_dia = resuestas[0, :] # [12, 8, 4]
chat = resuestas[:, 1] # [8, 7, 9, 10]
dos_primeros_dias = resuestas[:2, :]
```

La posición 1 solo significa chat porque `canales[1]` lo documenta. Si se reordena canales sin reordenar las columnas, el código seguirá funcionando y la conclusión será errónea. En tablas de producción, Pandas reduce este riesgo al usar etiquetas; aun así hay que validar las claves.

Error habitual: forma válida, comparación inválida

Dos arrays pueden tener misma forma y referirse a períodos distintos. Restar las solicitudes de lunes a jueves a las de viernes a lunes entrega cuatro números, pero no mide una evolución comparable si los días no representan la misma condición. La forma comprueba compatibilidad técnica; no comprueba comparabilidad de negocio.

Resumen y comprobación

- Documenta qué representa cada dimensión antes de usar `shape`.
- `axis=0` devuelve una medida por columna; `axis=1`, una medida por fila en este contrato.
- Los índices son posiciones, no etiquetas con significado propio.

Pregunta: si `resuestas.mean(axis=0)` tiene forma `(3,)`, ¿qué representa cada resultado?
Continúa con máscaras para seleccionar por una condición explícita.

Máscaras, valores ausentes y copias

Objetivos y prerequisites

Sabrás construir una condición, usarla para seleccionar datos, reconocer un valor ausente y evitar modificar un array por accidente. Requiere entender shape, índices y operaciones de comparación.

De una pregunta a una máscara

La responsable de soporte pregunta: «¿qué días superaron 50 minutos medios en chat?». Una **máscara booleana** es un array de True y False, uno por valor, que expresa esa pregunta:

```
chat_minutos = np.array([48.0, 55.0, 51.0, 43.0])
supera_objetivo = chat_minutos > 50
print(supera_objetivo)           # [False, True, True, False]
print(chat_minutos[supera_objetivo]) # [55., 51.]
```

El umbral de 50 no lo inventa NumPy. Debe proceder de un acuerdo de nivel de servicio, un objetivo o una hipótesis explícita. Cambiarlo a 45 cambia la población seleccionada y, por tanto, la historia que se cuenta.

Para combinar requisitos usa & (y) y | (o), siempre entre paréntesis:

```
es_lento = chat_minutos > 50
es_critico = chat_minutos >= 60
revisar = es_lento & ~es_critico
```

No escribas `es_lento and es_critico`: and pregunta por el array completo y no representa una comparación elemento a elemento.

Ausencia no es cero

Un **valor ausente** significa que no conocemos o no recibimos el valor. En datos numéricos NumPy suele representarlo con `np.nan` (*not a number*). No equivale a cero: cero minutos sería una medida real que hay que interpretar; NaN dice que no hay medida utilizable.

```
tiempo_chat = np.array([48.0, np.nan, 51.0, 43.0])
print(np.isnan(tiempo_chat))      # identifica el hueco
print(np.mean(tiempo_chat))      # nan: la ausencia se propaga
print(np.nanmean(tiempo_chat))   # 47.33..., ignora NaN
```

`np.nanmean` puede ser apropiado para un resumen exploratorio, pero no «arregla» el problema. Primero pregunta por qué faltó la medición: ¿falló el tracking, no hubo conversaciones o se retrasó la carga? Si los días de más tráfico son precisamente los ausentes, ignorarlos sesga el resultado.

Vista frente a copia: una modificación con consecuencias

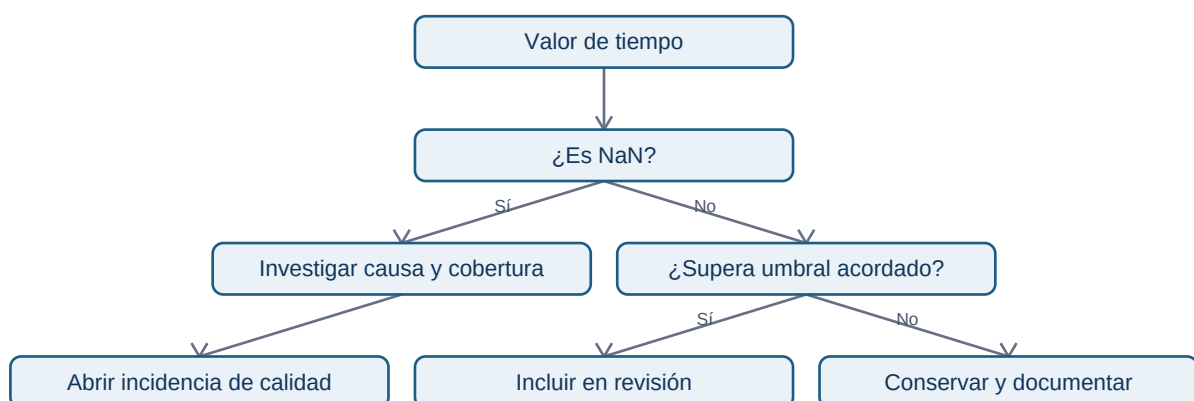
Un corte básico suele devolver una **vista**: comparte memoria con el array original. Una copia tiene memoria propia. La diferencia importa cuando limpias o pruebas transformaciones:

```
original = np.array([48.0, 55.0, 51.0, 43.0])
vista = original[:2]
vista[0] = 999.0
print(original[0]) # 999.0: la vista modificó el original

seguro = original.copy()
seguro[0] = 48.0 # solo cambia seguro
```

No dependas de recordar todas las reglas de indexación avanzada. Si vas a alterar valores para una prueba o una imputación, llama explícitamente a `.copy()` y conserva el origen. Esto permite repetir la auditoría.

Flujo de decisión para un dato sospechoso



El flujo separa ausencia de rendimiento bajo: sustituir ambos por cero convertiría problemas distintos en el mismo número.

Resumen y comprobación

- Una máscara deja visible el criterio de selección; valida su longitud y procedencia.
- NaN es desconocido, no un cero conveniente.
- Copia antes de modificar datos para análisis; documenta cualquier regla de tratamiento.

¿Qué respuesta adicional pedirías antes de usar `np.nanmean` para informar a dirección? En la siguiente lección aplicarás reglas distintas por canal sin duplicar la matriz.

Broadcasting y reglas por canal

Objetivos y prerequisites

Aprenderás cuándo NumPy puede combinar dimensiones de forma segura y cómo detectar una regla aplicada sobre el eje equivocado. Requiere matrices, `shape` y operaciones vectorizadas.

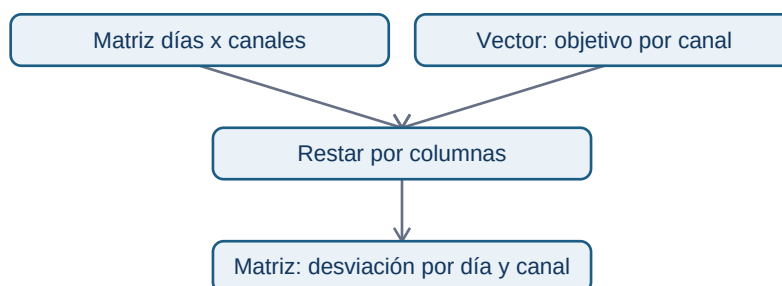
La necesidad: un objetivo distinto para cada canal

NexoCloud pacta objetivos de respuesta diferentes: web 40 minutos, chat 45 y correo 120. Los tiempos diarios tienen forma (días, canales); el objetivo tiene forma (canales,):

```
tiempos = np.array([
    [38.0, 52.0, 110.0],
    [41.0, 43.0, 130.0],
    [44.0, 47.0, 115.0],
])
objetivos = np.array([40.0, 45.0, 120.0])
desviacion = tiempos - objetivos
```

Broadcasting es el conjunto de reglas por el que NumPy alinea dimensiones compatibles. Aquí interpreta `objetivos` como una fila que puede usarse para cada día, sin que tengas que crear manualmente tres copias. El resultado (3, 3) conserva una desviación por día y canal.

La pregunta es «¿qué objetivo se resta de cada celda?»:



Cada columna recibe su objetivo. No se está calculando todavía la causa de las demoras: solo una diferencia respecto a una referencia acordada.

Compatibilidad, no telepatía

NumPy compara dimensiones desde el final. Son compatibles si son iguales o una vale 1. Por eso (3, 3) y (3,) funcionan. En cambio, un vector de tres objetivos de día podría tener también forma (3,); NumPy no puede saber si representa días o canales. Si ambos tamaños coinciden, el código puede ejecutarse sobre el eje incorrecto.

Cuando el significado sea «un valor por fila», haz la orientación visible con `[:, np.newaxis]`:

```
factor_por_dia = np.array([1.0, 1.1, 0.9])[:, np.newaxis] # forma (3, 1)
ajustado = tiempos * factor_por_dia
```

`np.newaxis` agrega una dimensión de tamaño uno. No crea conocimiento de negocio; hace explícita la intención de multiplicar cada fila por su factor.

Error habitual: confundir eficiencia con corrección

Broadcasting ahorra código, pero puede amplificar un supuesto erróneo. Aplicar un factor de campaña a todos los canales cuando solo afectó a web produce números plausibles y falsos. Antes de ejecutar, escribe el contrato: forma, unidades, orden de canales y período de validez de la regla.

Resumen y comprobación

- Broadcasting combina dimensiones compatibles; no decide qué dimensión tiene sentido.
- Un vector (canales,) se alinea con la última dimensión de (días, canales).
- Usa (días, 1) para declarar una regla por fila y verifica el resultado.

¿Qué shape debería tener una tasa distinta para cada día y canal? Respuesta: (días, canales), salvo que una regla común esté justificada.

Simulación, reproducibilidad y laboratorio

Objetivos y prerequisites

Aplicarás el caso NexoCloud de principio a fin y sabrás para qué sirve fijar una semilla aleatoria. Requiere máscaras, NaN, ejes y broadcasting.

Simular no es observar

Una simulación crea datos artificiales según reglas elegidas. Sirve para practicar, explorar escenarios y comprobar código cuando aún no tienes acceso a producción. No permite afirmar que clientes reales se comportaron así: esa afirmación requeriría datos observados y evaluación de calidad.

```
generador = np.random.default_rng(2026)
ruido = generador.normal(loc=0, scale=4, size=(7, 3))
```

La **semilla** 2026 inicializa el generador de forma repetible: otra persona que ejecute las mismas instrucciones obtiene el mismo ruido. No convierte la simulación en verdadera ni debe reutilizarse como mecanismo de seguridad.

Laboratorio: responder una pregunta operativa

El script [04-operaciones-nexocloud.py](#) construye siete días de tiempos medios, incorpora un dato ausente y contesta:

1. ¿Qué días y canales superaron su objetivo?
2. ¿Qué media por canal puede calcularse sin ocultar la cobertura?
3. ¿Qué canal merece una revisión y qué dato faltante debe investigarse?

Antes de ejecutar, predice la forma de tiempos, objetivos y desviación. Después compara tu predicción con la salida. Un análisis reproducible deja visibles los datos de entrada, la semilla si se usa, las transformaciones y las decisiones; no solo un número final.

Una recomendación responsable

Si chat supera el objetivo en varios días, una recomendación inicial podría ser «revisar capacidad y clasificación de conversaciones de chat». No sería correcto afirmar que falta personal solo por esa matriz: pueden intervenir severidad, cambios de tracking, mezcla de casos o fallos de medición. NumPy prepara la evidencia numérica; la investigación operativa exige contexto.

Cierre

Has pasado de valores sueltos a una matriz con contrato, filtros, valores ausentes y reglas por canal. Resuelve ahora el [ejercicio aplicado](#). El bloque 05 trasladará este razonamiento a datos tabulares con columnas y claves.